Management System ISO 9001:2015 CERTIFIED
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

Changing Queue Methods in Java

Name: _____
Date: _____
Class Section: _____

Objective

In this activity, you will rewrite three basic Java Queue programs. You will change the methods that add, look at, and remove items to see how different Queue implementations react when they run out of items.

Problem Instructions

1. Review the three original Queue programs provided in our lesson **(LinkedList, PriorityQueue, and ArrayDeque).**
2. Rewrite each program by swapping out the methods using this guide:
   - **Instead of .offer(), use .add()**
   - **Instead of .peek(), use .element()**
   - **Instead of .poll(), use .remove()**
3. Type out, compile, and run your rewritten programs.
4. Answer the Reflective Questions at the end based on your results.

GENERAL INSTRUCTIONS
For each problem, follow these rules:

1. Import Queue Implementations: Your program must use the necessary Java utility imports:
2. Use your name in the class name: The class name must follow this format: QueueProb_FirstnameLastname
   - *For example, if the student's name is Juan Dela Cruz:* QueueProb_JuanDelaCruz.java
3. File name: The Java file must have exactly the same name as the public class name.
4. Use comments: Students should add inline comments to identify important queue operations (such as initialization, enqueueing, dequeueing, and inspecting elements).
5. Submit: For each problem, students should submit:
   - **.java source code**
   - **Screenshot of the program output**
   - **Answers to the reflective questions**

Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

Expected Output

Task 1: Standard Line (LinkedList)

```
Initial Queue: [Alice, Bob, Charlie]
Next person to be served: Alice
Served: Alice
Queue after serving: [Bob, Charlie]
```

Task 2: Priority Line (PriorityQueue)

```
Processing Priority Queue:
Removed: 10
Removed: 20
Removed: 30
Removed: 40
```

Task 3: Printer Line (ArrayDeque)

```
Current Print Queue: [Tax_Return_2026.pdf, Quarterly_Report.docx, Family

Next document to print: Tax_Return_2026.pdf

--- Starting Printing Process ---
Printing: Tax_Return_2026.pdf ... Done!
Printing: Quarterly_Report.docx ... Done!
Printing: Family_Photo.jpg ... Done!

Is the print queue empty? true
```

Reflective Questions

1. When you ran Task 1 (LinkedList), what specific error message or exception appeared in your console at the very end of the program?

2. Why did Task 2 (PriorityQueue) and Task 3 (ArrayDeque) finish running successfully without crashing, even though they used .remove()? *(Hint: Look closely at the while loops used in those tasks).*

3. Based on what you observed, what is the main risk of using .remove() and .element() instead of .poll() and .peek()?

Grading Rubric

Criteria	Excellent (5 pts)	Good (3-4 pts)	Needs Improvement (1-2 pts)
Code Correctness	All 3 programs are modified correctly with no typos.	1 or 2 programs have minor typos but the concept is understood.	Code does not compile or methods were not swapped correctly.
Execution	Student successfully ran all programs and safely triggered the Task 1 crash.	Student ran the programs but did not understand the output.	Programs were not executed.
Reflective Answers	Questions are answered thoroughly, demonstrating clear understanding of exceptions.	Answers are brief but mostly correct.	Answers are missing or completely incorrect.